PROYECTO ACTIVA

Código completo y configuración

Aplicación de consola para gestionar actividades y reservas

Contenido organizado por partes y archivos

Generado a partir del contenido actual del proyecto.

Incluye fuentes Java, persistencia, compilación, ejecución y guía.

https://github.com/Vil-Blass/TareaEveUD4.git

ÍNDICE DE CONTENIDOS

README.md	3
pom.xml	5
Dockerfile	6
compose.yaml	7
.env.example	8
.gitignore	9
.dockerignore	10
src/main/resources/META-INF/persistence.xml	11
src/main/java/es/activa/config/PersistenceConfig.java	12
src/main/java/es/activa/domain/Actividad.java	13
src/main/java/es/activa/domain/Cliente.java	15
src/main/java/es/activa/domain/Monitor.java	16
src/main/java/es/activa/domain/Reserva.java	17
src/main/java/es/activa/service/ActivaService.java	18
src/main/java/es/activa/main/ActivaApplication.java	23

PARTE: README.md

README.md

````
# Activa: reservas de actividades guiadas

Aplicación Java de consola para gestionar monitores, actividades, clientes y reservas con JPA y Hibernate
.

## Requisitos

- Java 25 (LTS)
- Maven 3.9 o posterior
- MySQL 8 y el esquema `dbactiva` preparado mediante el script `dbactiva.sql` proporcionado por la tarea

El proyecto utiliza Hibernate ORM 6.4.10.Final y el conector MySQL 8.4.0. No incluye ni genera el script
de tablas. Hibernate está configurado con `hibernate.hbm2ddl.auto=validate`: comprueba el esquema existen
te y no lo crea ni lo modifica.

## Conexión

La conexión predeterminada es `jdbc:mysql://localhost:3306/dbactiva`, con el usuario `root`. Configura la
 contraseña mediante `DB_PASSWORD`; también se pueden cambiar la URL y el usuario. No guardes credenciale
s reales en archivos versionados:

- `DB_URL`: URL JDBC de la base de datos
- `DB_USER`: usuario
- `DB_PASSWORD`: contraseña

En PowerShell, establece la contraseña solo en la sesión actual antes de ejecutar la aplicación:

```powershell
$secret = Read-Host "Contraseña de MySQL" -AsSecureString
$pointer = [Runtime.InteropServices.Marshal]::SecureStringToBSTR($secret)
try {
    $env:DB_PASSWORD = [Runtime.InteropServices.Marshal]::PtrToStringBSTR($pointer)
    mvn exec:java
} finally {
    Remove-Item Env:DB_PASSWORD -ErrorAction SilentlyContinue
    [Runtime.InteropServices.Marshal]::ZeroFreeBSTR($pointer)
}
```

## Ejecución

Desde la carpeta del proyecto, ejecuta:

```text
mvn clean compile
mvn exec:java
```

El menú permanece activo hasta elegir la opción 10. Las opciones 8 y 9 permiten modificar y eliminar moni
tores, actividades y clientes. No se puede borrar un monitor con actividades ni una actividad o cliente c
on reservas; primero hay que reasignar o eliminar las referencias dependientes, cancelando las reservas d
esde la opción 6. Para buscar actividades se introduce la fecha como `dd/MM/yyyy`; para crear o modificar
 una actividad, como `dd/MM/yyyy HH:mm`.

## Ejecución con Docker

Requiere Docker Desktop con Docker Compose. La configuración levanta MySQL 8.4 y la aplicación Java 25. E
l volumen `mysql-data` conserva la base de datos entre reinicios. Hibernate solo valida el esquema (`vali
date`); no crea tablas.

1. Crea el fichero local de variables, que está excluido del control de versiones, y ábrelo para reemplaz
ar la contraseña de ejemplo por la contraseña configurada en MySQL. Compose necesita este archivo para in
terpolar `MYSQL_ROOT_PASSWORD`:

   ```powershell
   Copy-Item .env.example .env
   notepad .env
   ```

   Deja la variable con este formato, usando la contraseña real de MySQL:

   ```text
   MYSQL_ROOT_PASSWORD=tu_contraseña_de_root
   ```

2. Inicia MySQL:
````

PARTE: README.md (continuación)

````
   ```powershell
   docker compose up -d db
   ```

3. Copia el `dbactiva.sql` entregado con la tarea a la carpeta del proyecto e impórtalo una sola vez en l
a base MySQL del contenedor:

   ```powershell
   docker compose cp .\dbactiva.sql db:/tmp/dbactiva.sql
   docker compose exec db sh -c 'mysql -uroot -p"$MYSQL_ROOT_PASSWORD" < /tmp/dbactiva.sql'
   ```

4. Construye la imagen y ejecuta la aplicación interactiva:

   ```powershell
   docker compose --profile app run --build --rm app
   ```

Elige la opción 10 para salir. Para volver a iniciar la aplicación, repite el paso 4; no vuelvas a import
ar el script. `docker compose down` detiene y elimina los contenedores, pero conserva el volumen de datos
.

## Organización

- `domain`: entidades JPA `Monitor`, `Actividad`, `Cliente` y `Reserva`.
- `service`: reglas de negocio, consultas JPQL y transacciones.
- `config`: creación de la fábrica `EntityManagerFactory`.
- `main`: menú e interacción con consola.

La reserva y la cancelación usan un único `EntityManager` y una transacción por operación. Si se produce
una excepción, se ejecuta rollback antes de cerrar el gestor.

## Uso de IA

Se utilizó un asistente de IA como apoyo para estructurar e implementar el proyecto a partir del enunciad
o. La persona que entrega el trabajo debe revisar el código, probarlo contra la base de datos proporciona
da y explicar las decisiones tomadas; esta nota no sustituye la autoevaluación ni las evidencias requerid
as por la tarea.
````

PARTE: pom.xml

pom.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.apache.org/xsd/maven-4.0.0.x
sd">
    <modelVersion>4.0.0</modelVersion>

    <groupId>es.activa</groupId>
    <artifactId>activa-reservas</artifactId>
    <version>1.0.0</version>

    <properties>
        <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
        <maven.compiler.release>25</maven.compiler.release>
        <hibernate.version>6.4.10.Final</hibernate.version>
    </properties>

    <dependencies>
        <dependency>
            <groupId>org.hibernate.orm</groupId>
            <artifactId>hibernate-core</artifactId>
            <version>${hibernate.version}</version>
        </dependency>
        <dependency>
            <groupId>com.mysql</groupId>
            <artifactId>mysql-connector-j</artifactId>
            <version>8.4.0</version>
            <scope>runtime</scope>
        </dependency>
        <dependency>
            <groupId>org.slf4j</groupId>
            <artifactId>slf4j-simple</artifactId>
            <version>2.0.13</version>
            <scope>runtime</scope>
        </dependency>
    </dependencies>

    <build>
        <plugins>
            <plugin>
                <groupId>org.apache.maven.plugins</groupId>
                <artifactId>maven-compiler-plugin</artifactId>
                <version>3.13.0</version>
            </plugin>
            <plugin>
                <groupId>org.codehaus.mojo</groupId>
                <artifactId>exec-maven-plugin</artifactId>
                <version>3.3.0</version>
                <configuration>
                    <mainClass>es.activa.main.ActivaApplication</mainClass>
                </configuration>
            </plugin>
        </plugins>
    </build>
</project>
```

PARTE: Dockerfile

Dockerfile

```
FROM maven:3.9-eclipse-temurin-25 AS build

WORKDIR /workspace

COPY pom.xml .
COPY src ./src

RUN mvn -B -DskipTests package dependency:copy-dependencies \
    -DincludeScope=runtime \
    -DoutputDirectory=/workspace/target/dependency

FROM eclipse-temurin:25-jre

WORKDIR /app

COPY --from=build --chown=10001:10001 /workspace/target/classes ./classes
COPY --from=build --chown=10001:10001 /workspace/target/dependency/ ./lib/

USER 10001:10001

ENTRYPOINT ["java", "-cp", "classes:lib/*", "es.activa.main.ActivaApplication"]
```

PARTE: compose.yaml

compose.yaml

```
services:
  db:
    image: mysql:8.4
    environment:
      MYSQL_ROOT_PASSWORD: ${MYSQL_ROOT_PASSWORD:?Define MYSQL_ROOT_PASSWORD in .env}
    volumes:
      - mysql-data:/var/lib/mysql
    healthcheck:
      test: ["CMD-SHELL", "mysqladmin ping -h 127.0.0.1 -uroot -p\"$$MYSQL_ROOT_PASSWORD\" --silent"]
      interval: 5s
      timeout: 5s
      retries: 20
      start_period: 30s

  app:
    profiles: ["app"]
    build:
      context: .
      dockerfile: Dockerfile
    environment:
      DB_URL: jdbc:mysql://db:3306/dbactiva?serverTimezone=UTC
      DB_USER: root
      DB_PASSWORD: ${MYSQL_ROOT_PASSWORD:?Define MYSQL_ROOT_PASSWORD in .env}
    depends_on:
      db:
        condition: service_healthy
    stdin_open: true
    tty: true

volumes:
  mysql-data:
```

PARTE: .env.example

.env.example

```
# Change this value before starting the containers.
MYSQL_ROOT_PASSWORD=change-this-password
```

PARTE: .gitignore

.gitignore

```
target/
.idea/
.vscode/
*.iml
.env
```

PARTE: .dockerignore

.dockerignore

```
.git
.idea
.vscode
target
.env
*.iml
*.docx
*.docx#
```

PARTE: src/main/resources/META-INF/persistence.xml

src/main/resources/META-INF/persistence.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<persistence xmlns="https://jakarta.ee/xml/ns/persistence"
             xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
             xsi:schemaLocation="https://jakarta.ee/xml/ns/persistence https://jakarta.ee/xml/ns/persiste
nce/persistence_3_0.xsd"
             version="3.0">
    <persistence-unit name="activaPU" transaction-type="RESOURCE_LOCAL">
        <provider>org.hibernate.jpa.HibernatePersistenceProvider</provider>
        <class>es.activa.domain.Monitor</class>
        <class>es.activa.domain.Actividad</class>
        <class>es.activa.domain.Cliente</class>
        <class>es.activa.domain.Reserva</class>
        <properties>
            <property name="jakarta.persistence.jdbc.driver" value="com.mysql.cj.jdbc.Driver"/>
            <property name="jakarta.persistence.jdbc.url" value="jdbc:mysql://localhost:3306/dbactiva?ser
verTimezone=UTC"/>
            <property name="jakarta.persistence.jdbc.user" value="root"/>
            <property name="jakarta.persistence.jdbc.password" value=""/>
            <property name="hibernate.hbm2ddl.auto" value="validate"/>
            <property name="hibernate.show_sql" value="false"/>
        </properties>
    </persistence-unit>
</persistence>
```

PARTE: src/main/java/es/activa/config/PersistenceConfig.java

src/main/java/es/activa/config/PersistenceConfig.java

```
package es.activa.config;

import jakarta.persistence.EntityManagerFactory;
import jakarta.persistence.Persistence;

import java.util.HashMap;
import java.util.Map;

public final class PersistenceConfig {
    private PersistenceConfig() {
    }

    public static EntityManagerFactory createEntityManagerFactory() {
        Map<String, Object> properties = new HashMap<>();
        putEnvironmentValue(properties, "DB_URL", "jakarta.persistence.jdbc.url");
        putEnvironmentValue(properties, "DB_USER", "jakarta.persistence.jdbc.user");
        putEnvironmentValue(properties, "DB_PASSWORD", "jakarta.persistence.jdbc.password");
        return Persistence.createEntityManagerFactory("activaPU", properties);
    }

    private static void putEnvironmentValue(Map<String, Object> properties, String variable, String prope
rty) {
        String value = System.getenv(variable);
        if (value != null) {
            properties.put(property, value);
        }
    }
}
```

PARTE: src/main/java/es/activa/domain/Actividad.java

src/main/java/es/activa/domain/Actividad.java

```
package es.activa.domain;

import jakarta.persistence.Column;
import jakarta.persistence.Entity;
import jakarta.persistence.FetchType;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import jakarta.persistence.JoinColumn;
import jakarta.persistence.ManyToOne;
import jakarta.persistence.Table;

import java.time.LocalDateTime;
import java.util.Objects;

@Entity
@Table(name = "actividad")
public class Actividad {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(name = "nombre", nullable = false)
    private String nombre;

    @Column(name = "localidad", nullable = false)
    private String localidad;

    @Column(name = "fecha_hora", nullable = false)
    private LocalDateTime fechaHora;

    @Column(name = "plazas_totales", nullable = false)
    private int plazasTotales;

    @Column(name = "plazas_disponibles", nullable = false)
    private int plazasDisponibles;

    @ManyToOne(fetch = FetchType.LAZY, optional = false)
    @JoinColumn(name = "monitor_id", nullable = false)
    private Monitor monitor;

    protected Actividad() {
    }

    public Actividad(String nombre, String localidad, LocalDateTime fechaHora, int plazasTotales, Monitor
 monitor) {
        this.nombre = nombre;
        this.localidad = localidad;
        this.fechaHora = fechaHora;
        this.plazasTotales = plazasTotales;
        this.plazasDisponibles = plazasTotales;
        setMonitor(monitor);
    }

    public void setMonitor(Monitor monitor) {
        Objects.requireNonNull(monitor, "La actividad debe tener un monitor.");
        if (this.monitor == monitor) {
            monitor.addActividadReference(this);
            return;
        }
        if (this.monitor != null) {
            this.monitor.removeActividadReference(this);
        }
        this.monitor = monitor;
        monitor.addActividadReference(this);
    }

    public void desvincularMonitorAlEliminar() {
        if (monitor != null) {
            Monitor monitorAnterior = monitor;
            monitor = null;
            monitorAnterior.removeActividadReference(this);
        }
    }

    public void actualizar(String nombre, String localidad, LocalDateTime fechaHora,
```

PARTE: src/main/java/es/activa/domain/Actividad.java (continuación)

```
                        int plazasTotales, Monitor monitor) {
        int plazasReservadas = this.plazasTotales - this.plazasDisponibles;
        if (plazasTotales < plazasReservadas) {
            throw new IllegalArgumentException(
                    "Las plazas totales no pueden ser inferiores a las plazas ya reservadas.");
        }
        this.nombre = nombre;
        this.localidad = localidad;
        this.fechaHora = fechaHora;
        this.plazasTotales = plazasTotales;
        this.plazasDisponibles = plazasTotales - plazasReservadas;
        setMonitor(monitor);
    }

    public void reservar(int numeroPlazas) {
        plazasDisponibles -= numeroPlazas;
    }

    public void liberar(int numeroPlazas) {
        plazasDisponibles += numeroPlazas;
    }

    public Long getId() {
        return id;
    }

    public String getNombre() {
        return nombre;
    }

    public String getLocalidad() {
        return localidad;
    }

    public LocalDateTime getFechaHora() {
        return fechaHora;
    }

    public int getPlazasTotales() {
        return plazasTotales;
    }

    public int getPlazasDisponibles() {
        return plazasDisponibles;
    }

    public Monitor getMonitor() {
        return monitor;
    }
}
```

PARTE: src/main/java/es/activa/domain/Cliente.java

src/main/java/es/activa/domain/Cliente.java

```
package es.activa.domain;

import jakarta.persistence.Column;
import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import jakarta.persistence.Table;

@Entity
@Table(name = "cliente")
public class Cliente {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(name = "nombre", nullable = false)
    private String nombre;

    @Column(name = "email", nullable = false, unique = true)
    private String email;

    protected Cliente() {
    }

    public Cliente(String nombre, String email) {
        this.nombre = nombre;
        this.email = email;
    }

    public void actualizar(String nombre, String email) {
        this.nombre = nombre;
        this.email = email;
    }

    public Long getId() {
        return id;
    }

    public String getNombre() {
        return nombre;
    }

    public String getEmail() {
        return email;
    }
}
```

PARTE: src/main/java/es/activa/domain/Monitor.java

src/main/java/es/activa/domain/Monitor.java

```
package es.activa.domain;

import jakarta.persistence.Column;
import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import jakarta.persistence.OneToMany;
import jakarta.persistence.Table;

import java.util.ArrayList;
import java.util.List;

@Entity
@Table(name = "monitor")
public class Monitor {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(name = "nombre", nullable = false)
    private String nombre;

    @Column(name = "especialidad", nullable = false)
    private String especialidad;

    @OneToMany(mappedBy = "monitor")
    private List<Actividad> actividades = new ArrayList<>();

    protected Monitor() {
    }

    public Monitor(String nombre, String especialidad) {
        this.nombre = nombre;
        this.especialidad = especialidad;
    }

    public void addActividad(Actividad actividad) {
        actividad.setMonitor(this);
    }

    void addActividadReference(Actividad actividad) {
        if (!actividades.contains(actividad)) {
            actividades.add(actividad);
        }
    }

    void removeActividadReference(Actividad actividad) {
        actividades.remove(actividad);
    }

    public void actualizar(String nombre, String especialidad) {
        this.nombre = nombre;
        this.especialidad = especialidad;
    }

    public Long getId() {
        return id;
    }

    public String getNombre() {
        return nombre;
    }

    public String getEspecialidad() {
        return especialidad;
    }

    public List<Actividad> getActividades() {
        return List.copyOf(actividades);
    }
}
```

PARTE: src/main/java/es/activa/domain/Reserva.java

src/main/java/es/activa/domain/Reserva.java

```
package es.activa.domain;

import jakarta.persistence.Column;
import jakarta.persistence.Entity;
import jakarta.persistence.FetchType;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import jakarta.persistence.JoinColumn;
import jakarta.persistence.ManyToOne;
import jakarta.persistence.Table;

import java.time.LocalDateTime;

@Entity
@Table(name = "reserva")
public class Reserva {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(name = "fecha_reserva", nullable = false)
    private LocalDateTime fechaReserva;

    @Column(name = "numero_plazas", nullable = false)
    private int numeroPlazas;

    @ManyToOne(fetch = FetchType.LAZY, optional = false)
    @JoinColumn(name = "actividad_id", nullable = false)
    private Actividad actividad;

    @ManyToOne(fetch = FetchType.LAZY, optional = false)
    @JoinColumn(name = "cliente_id", nullable = false)
    private Cliente cliente;

    protected Reserva() {
    }

    public Reserva(LocalDateTime fechaReserva, int numeroPlazas, Actividad actividad, Cliente cliente) {
        this.fechaReserva = fechaReserva;
        this.numeroPlazas = numeroPlazas;
        this.actividad = actividad;
        this.cliente = cliente;
    }

    public Long getId() {
        return id;
    }

    public int getNumeroPlazas() {
        return numeroPlazas;
    }

    public Actividad getActividad() {
        return actividad;
    }
}
```

PARTE: src/main/java/es/activa/service/ActivaService.java

src/main/java/es/activa/service/ActivaService.java

```
package es.activa.service;

import es.activa.domain.Actividad;
import es.activa.domain.Cliente;
import es.activa.domain.Monitor;
import es.activa.domain.Reserva;
import jakarta.persistence.EntityManager;
import jakarta.persistence.EntityManagerFactory;
import jakarta.persistence.EntityTransaction;

import java.time.LocalDate;
import java.time.LocalDateTime;
import java.util.List;
import java.util.Locale;
import java.util.function.Function;

public class ActivaService {
    private final EntityManagerFactory entityManagerFactory;

    public ActivaService(EntityManagerFactory entityManagerFactory) {
        this.entityManagerFactory = entityManagerFactory;
    }

    public Long crearMonitor(String nombre, String especialidad) {
        requireText(nombre, "El nombre");
        requireText(especialidad, "La especialidad");
        return inTransaction(entityManager -> {
            Monitor monitor = new Monitor(nombre.trim(), especialidad.trim());
            entityManager.persist(monitor);
            return monitor.getId();
        });
    }

    public void modificarMonitor(long monitorId, String nombre, String especialidad) {
        requireText(nombre, "El nombre");
        requireText(especialidad, "La especialidad");
        inTransaction(entityManager -> {
            Monitor monitor = entityManager.find(Monitor.class, monitorId);
            if (monitor == null) {
                throw new IllegalArgumentException("No existe un monitor con ese ID.");
            }
            monitor.actualizar(nombre.trim(), especialidad.trim());
            return null;
        });
    }

    public void eliminarMonitor(long monitorId) {
        inTransaction(entityManager -> {
            Monitor monitor = entityManager.find(Monitor.class, monitorId);
            if (monitor == null) {
                throw new IllegalArgumentException("No existe un monitor con ese ID.");
            }
            Long actividades = entityManager.createQuery(
                            "select count(a) from Actividad a where a.monitor.id = :monitorId", Long.clas
s)
                    .setParameter("monitorId", monitorId)
                    .getSingleResult();
            if (actividades > 0) {
                throw new IllegalArgumentException(
                        "No se puede eliminar el monitor porque tiene actividades asignadas.");
            }
            entityManager.remove(monitor);
            return null;
        });
    }

    public Long crearActividad(String nombre, String localidad, LocalDateTime fechaHora,
                               int plazasTotales, long monitorId) {
        requireText(nombre, "El nombre");
        requireText(localidad, "La localidad");
        if (fechaHora == null) {
            throw new IllegalArgumentException("La fecha y hora son obligatorias.");
        }
        if (plazasTotales <= 0) {
            throw new IllegalArgumentException("El número de plazas debe ser positivo.");
        }
```

PARTE: src/main/java/es/activa/service/ActivaService.java (continuación)

```
        return inTransaction(entityManager -> {
            Monitor monitor = entityManager.find(Monitor.class, monitorId);
            if (monitor == null) {
                throw new IllegalArgumentException("No existe un monitor con ese ID.");
            }
            Actividad actividad = new Actividad(
                    nombre.trim(), localidad.trim(), fechaHora, plazasTotales, monitor);
            monitor.addActividad(actividad);
            entityManager.persist(actividad);
            return actividad.getId();
        });
    }

    public void modificarActividad(long actividadId, String nombre, String localidad,
                                   LocalDateTime fechaHora, int plazasTotales, long monitorId) {
        requireText(nombre, "El nombre");
        requireText(localidad, "La localidad");
        if (fechaHora == null) {
            throw new IllegalArgumentException("La fecha y hora son obligatorias.");
        }
        if (plazasTotales <= 0) {
            throw new IllegalArgumentException("El número de plazas debe ser positivo.");
        }
        inTransaction(entityManager -> {
            Actividad actividad = entityManager.find(Actividad.class, actividadId);
            if (actividad == null) {
                throw new IllegalArgumentException("No existe una actividad con ese ID.");
            }
            Monitor monitor = entityManager.find(Monitor.class, monitorId);
            if (monitor == null) {
                throw new IllegalArgumentException("No existe un monitor con ese ID.");
            }
            actividad.actualizar(nombre.trim(), localidad.trim(), fechaHora, plazasTotales, monitor);
            return null;
        });
    }

    public void eliminarActividad(long actividadId) {
        inTransaction(entityManager -> {
            Actividad actividad = entityManager.find(Actividad.class, actividadId);
            if (actividad == null) {
                throw new IllegalArgumentException("No existe una actividad con ese ID.");
            }
            Long reservas = entityManager.createQuery(
                            "select count(r) from Reserva r where r.actividad.id = :actividadId", Long.cl
ass)
                    .setParameter("actividadId", actividadId)
                    .getSingleResult();
            if (reservas > 0) {
                throw new IllegalArgumentException(
                        "No se puede eliminar la actividad porque tiene reservas. Cancélalas primero.");
            }
            actividad.desvincularMonitorAlEliminar();
            entityManager.remove(actividad);
            return null;
        });
    }

    public List<Actividad> buscarDisponibles(String localidad, LocalDate fecha, int plazasNecesarias) {
        requireText(localidad, "La localidad");
        if (fecha == null) {
            throw new IllegalArgumentException("La fecha es obligatoria.");
        }
        if (plazasNecesarias <= 0) {
            throw new IllegalArgumentException("El número de plazas debe ser positivo.");
        }
        String localidadNormalizada = localidad.trim().toLowerCase(Locale.ROOT);
        LocalDateTime inicio = fecha.atStartOfDay();
        LocalDateTime fin = fecha.plusDays(1).atStartOfDay();
        return inEntityManager(entityManager -> entityManager.createQuery(
                        "select a from Actividad a join fetch a.monitor "
                                + "where lower(a.localidad) = :localidad "
                                + "and a.fechaHora >= :inicio and a.fechaHora < :fin "
                                + "and a.plazasDisponibles >= :plazas "
                                + "order by a.fechaHora",
                        Actividad.class)
                .setParameter("localidad", localidadNormalizada)
                .setParameter("inicio", inicio)
                .setParameter("fin", fin)
```

PARTE: src/main/java/es/activa/service/ActivaService.java (continuación)

```
                .setParameter("plazas", plazasNecesarias)
                .getResultList());
    }

    public Long crearCliente(String nombre, String email) {
        requireText(nombre, "El nombre");
        String emailNormalizado = validarEmail(email);
        String emailBusqueda = emailNormalizado.toLowerCase(Locale.ROOT);
        return inTransaction(entityManager -> {
            Long duplicados = entityManager.createQuery(
                            "select count(c) from Cliente c where lower(trim(c.email)) = :email", Long.cl
ass)
                    .setParameter("email", emailBusqueda)
                    .getSingleResult();
            if (duplicados > 0) {
                throw new IllegalArgumentException("Ya existe un cliente con ese correo electrónico.");
            }
            Cliente cliente = new Cliente(nombre.trim(), emailNormalizado);
            entityManager.persist(cliente);
            return cliente.getId();
        });
    }

    public void modificarCliente(long clienteId, String nombre, String email) {
        requireText(nombre, "El nombre");
        String emailNormalizado = validarEmail(email);
        String emailBusqueda = emailNormalizado.toLowerCase(Locale.ROOT);
        inTransaction(entityManager -> {
            Cliente cliente = entityManager.find(Cliente.class, clienteId);
            if (cliente == null) {
                throw new IllegalArgumentException("No existe un cliente con ese ID.");
            }
            Long duplicados = entityManager.createQuery(
                            "select count(c) from Cliente c "
                                    + "where c.id <> :clienteId and lower(trim(c.email)) = :email",
                            Long.class)
                    .setParameter("clienteId", clienteId)
                    .setParameter("email", emailBusqueda)
                    .getSingleResult();
            if (duplicados > 0) {
                throw new IllegalArgumentException("Ya existe un cliente con ese correo electrónico.");
            }
            cliente.actualizar(nombre.trim(), emailNormalizado);
            return null;
        });
    }

    public void eliminarCliente(long clienteId) {
        inTransaction(entityManager -> {
            Cliente cliente = entityManager.find(Cliente.class, clienteId);
            if (cliente == null) {
                throw new IllegalArgumentException("No existe un cliente con ese ID.");
            }
            Long reservas = entityManager.createQuery(
                            "select count(r) from Reserva r where r.cliente.id = :clienteId", Long.class)
                    .setParameter("clienteId", clienteId)
                    .getSingleResult();
            if (reservas > 0) {
                throw new IllegalArgumentException(
                        "No se puede eliminar el cliente porque tiene reservas. Cancélalas primero.");
            }
            entityManager.remove(cliente);
            return null;
        });
    }

    public ReservaCreada crearReserva(long clienteId, long actividadId, int numeroPlazas) {
        if (numeroPlazas <= 0) {
            throw new IllegalArgumentException("El número de plazas debe ser positivo.");
        }
        return inTransaction(entityManager -> {
            Cliente cliente = entityManager.find(Cliente.class, clienteId);
            if (cliente == null) {
                throw new IllegalArgumentException("No existe un cliente con ese ID.");
            }
            Actividad actividad = entityManager.find(Actividad.class, actividadId);
            if (actividad == null) {
                throw new IllegalArgumentException("No existe una actividad con ese ID.");
            }
```

PARTE: src/main/java/es/activa/service/ActivaService.java (continuación)

```
            if (actividad.getPlazasDisponibles() < numeroPlazas) {
                throw new IllegalArgumentException("No hay plazas suficientes para esta reserva.");
            }
            actividad.reservar(numeroPlazas);
            Reserva reserva = new Reserva(LocalDateTime.now(), numeroPlazas, actividad, cliente);
            entityManager.persist(reserva);
            entityManager.flush();
            return new ReservaCreada(reserva.getId(), actividad.getPlazasDisponibles());
        });
    }

    public CancelacionRealizada cancelarReserva(long reservaId) {
        return inTransaction(entityManager -> {
            Reserva reserva = entityManager.find(Reserva.class, reservaId);
            if (reserva == null) {
                throw new IllegalArgumentException("No existe una reserva con ese ID.");
            }
            Actividad actividad = reserva.getActividad();
            int plazasDevueltas = reserva.getNumeroPlazas();
            actividad.liberar(plazasDevueltas);
            entityManager.remove(reserva);
            return new CancelacionRealizada(plazasDevueltas, actividad.getPlazasDisponibles());
        });
    }

    public List<Actividad> listarActividades() {
        return inEntityManager(entityManager -> entityManager.createQuery(
                        "select a from Actividad a join fetch a.monitor order by a.fechaHora",
                        Actividad.class)
                .getResultList());
    }

    private <T> T inTransaction(Function<EntityManager, T> operation) {
        EntityManager entityManager = entityManagerFactory.createEntityManager();
        EntityTransaction transaction = entityManager.getTransaction();
        try {
            transaction.begin();
            T result = operation.apply(entityManager);
            transaction.commit();
            return result;
        } catch (RuntimeException exception) {
            if (transaction.isActive()) {
                try {
                    transaction.rollback();
                } catch (RuntimeException rollbackException) {
                    exception.addSuppressed(rollbackException);
                }
            }
            throw exception;
        } finally {
            entityManager.close();
        }
    }

    private <T> T inEntityManager(Function<EntityManager, T> operation) {
        EntityManager entityManager = entityManagerFactory.createEntityManager();
        try {
            return operation.apply(entityManager);
        } finally {
            entityManager.close();
        }
    }

    private static void requireText(String value, String field) {
        if (value == null || value.isBlank()) {
            throw new IllegalArgumentException(field + " no puede estar vacío.");
        }
    }

    private static String validarEmail(String email) {
        requireText(email, "El email");
        String emailNormalizado = email.trim();
        if (!emailNormalizado.matches("^[^\\s@]+@[^\\s@]+\\.[^\\s@]+$")) {
            throw new IllegalArgumentException("El formato del correo electrónico no es válido.");
        }
        return emailNormalizado;
    }

    public record ReservaCreada(Long id, int plazasDisponibles) {
```

PARTE: src/main/java/es/activa/service/ActivaService.java (continuación)

```
    }

    public record CancelacionRealizada(int plazasDevueltas, int plazasDisponibles) {
    }
}
```

PARTE: src/main/java/es/activa/main/ActivaApplication.java

src/main/java/es/activa/main/ActivaApplication.java

```
package es.activa.main;

import es.activa.config.PersistenceConfig;
import es.activa.domain.Actividad;
import es.activa.service.ActivaService;
import jakarta.persistence.EntityManagerFactory;
import jakarta.persistence.PersistenceException;

import java.time.LocalDate;
import java.time.LocalDateTime;
import java.time.format.DateTimeFormatter;
import java.time.format.ResolverStyle;
import java.time.format.DateTimeParseException;
import java.util.Scanner;

public class ActivaApplication {
    private static final DateTimeFormatter DATE_TIME_FORMAT =
            DateTimeFormatter.ofPattern("dd/MM/uuuu HH:mm").withResolverStyle(ResolverStyle.STRICT);
    private static final DateTimeFormatter DATE_FORMAT =
            DateTimeFormatter.ofPattern("dd/MM/uuuu").withResolverStyle(ResolverStyle.STRICT);

    private final Scanner scanner;
    private final ActivaService service;

    private ActivaApplication(ActivaService service, Scanner scanner) {
        this.service = service;
        this.scanner = scanner;
    }

    public static void main(String[] args) {
        try (EntityManagerFactory factory = PersistenceConfig.createEntityManagerFactory();
             Scanner scanner = new Scanner(System.in)) {
            new ActivaApplication(new ActivaService(factory), scanner).run();
        } catch (PersistenceException | IllegalStateException exception) {
            System.err.println("No se pudo iniciar o utilizar la base de datos: " + usefulMessage(excepti
on));
        }
    }

    private void run() {
        boolean running = true;
        while (running) {
            printMenu();
            String option = readLine("Opción: ");
            if (option == null) {
                break;
            }
            try {
                switch (option.trim()) {
                    case "1" -> crearMonitor();
                    case "2" -> crearActividad();
                    case "3" -> buscarDisponibles();
                    case "4" -> crearCliente();
                    case "5" -> crearReserva();
                    case "6" -> cancelarReserva();
                    case "7" -> listarActividades();
                    case "8" -> modificarRegistro();
                    case "9" -> eliminarRegistro();
                    case "10" -> running = false;
                    default -> System.out.println("Opción incorrecta. Elige un número del 1 al 10.");
                }
            } catch (IllegalArgumentException exception) {
                System.out.println("Operación no realizada: " + usefulMessage(exception));
            } catch (RuntimeException exception) {
                System.out.println("Error al realizar la operación: " + usefulMessage(exception));
            }
        }
        System.out.println("Hasta pronto.");
    }

    private void crearMonitor() {
        String nombre = readRequiredText("Nombre: ");
        String especialidad = readRequiredText("Especialidad: ");
        Long id = service.crearMonitor(nombre, especialidad);
        System.out.println("Monitor creado. ID: " + id);
    }
```

PARTE: src/main/java/es/activa/main/ActivaApplication.java (continuación)

```
    private void crearActividad() {
        String nombre = readRequiredText("Nombre: ");
        String localidad = readRequiredText("Localidad: ");
        LocalDateTime fechaHora = readDateTime("Fecha y hora (dd/MM/yyyy HH:mm): ");
        int plazas = readPositiveInt("Plazas totales: ");
        long monitorId = readPositiveLong("ID del monitor: ");
        Long id = service.crearActividad(nombre, localidad, fechaHora, plazas, monitorId);
        System.out.println("Actividad creada. ID: " + id);
    }

    private void buscarDisponibles() {
        String localidad = readRequiredText("Localidad: ");
        LocalDate fecha = readDate("Día (dd/MM/yyyy): ");
        int plazas = readPositiveInt("Plazas necesarias: ");
        var actividades = service.buscarDisponibles(localidad, fecha, plazas);
        if (actividades.isEmpty()) {
            System.out.println("No hay actividades disponibles que coincidan.");
            return;
        }
        actividades.forEach(this::printActividadDisponible);
    }

    private void crearCliente() {
        String nombre = readRequiredText("Nombre: ");
        String email = readRequiredText("Email: ");
        Long id = service.crearCliente(nombre, email);
        System.out.println("Cliente creado. ID: " + id);
    }

    private void crearReserva() {
        long clienteId = readPositiveLong("ID del cliente: ");
        long actividadId = readPositiveLong("ID de la actividad: ");
        int plazas = readPositiveInt("Número de plazas: ");
        ActivaService.ReservaCreada reserva = service.crearReserva(clienteId, actividadId, plazas);
        System.out.println("Reserva confirmada. ID: " + reserva.id()
                + ". Plazas disponibles: " + reserva.plazasDisponibles());
    }

    private void cancelarReserva() {
        long reservaId = readPositiveLong("ID de la reserva: ");
        ActivaService.CancelacionRealizada cancelacion = service.cancelarReserva(reservaId);
        System.out.println("Reserva cancelada. Plazas recuperadas: " + cancelacion.plazasDevueltas()
                + ". Plazas disponibles: " + cancelacion.plazasDisponibles());
    }

    private void listarActividades() {
        var actividades = service.listarActividades();
        if (actividades.isEmpty()) {
            System.out.println("No hay actividades registradas.");
            return;
        }
        actividades.forEach(actividad -> System.out.printf(
                "ID: %d | %s | %s | %s | Monitor: %s | Plazas totales: %d | Disponibles: %d%n",
                actividad.getId(), actividad.getNombre(), actividad.getLocalidad(),
                actividad.getFechaHora().format(DATE_TIME_FORMAT), actividad.getMonitor().getNombre(),
                actividad.getPlazasTotales(), actividad.getPlazasDisponibles()));
    }

    private void modificarRegistro() {
        String tipo = readRecordType("modificar");
        long id = readPositiveLong("ID: ");
        switch (tipo) {
            case "1" -> {
                service.modificarMonitor(id, readRequiredText("Nombre: "),
                        readRequiredText("Especialidad: "));
                System.out.println("Monitor modificado.");
            }
            case "2" -> {
                String nombre = readRequiredText("Nombre: ");
                String localidad = readRequiredText("Localidad: ");
                LocalDateTime fechaHora = readDateTime("Fecha y hora (dd/MM/yyyy HH:mm): ");
                int plazas = readPositiveInt("Plazas totales: ");
                long monitorId = readPositiveLong("ID del monitor: ");
                service.modificarActividad(id, nombre, localidad, fechaHora, plazas, monitorId);
                System.out.println("Actividad modificada.");
            }
            case "3" -> {
                service.modificarCliente(id, readRequiredText("Nombre: "), readRequiredText("Email: "));
```

PARTE: src/main/java/es/activa/main/ActivaApplication.java (continuación)

```
                System.out.println("Cliente modificado.");
            }
            default -> throw new IllegalStateException("Tipo de registro no reconocido.");
        }
    }

    private void eliminarRegistro() {
        String tipo = readRecordType("eliminar");
        long id = readPositiveLong("ID: ");
        if (!confirmar("¿Confirmas la eliminación? (S/N): ")) {
            System.out.println("Eliminación cancelada.");
            return;
        }
        switch (tipo) {
            case "1" -> {
                service.eliminarMonitor(id);
                System.out.println("Monitor eliminado.");
            }
            case "2" -> {
                service.eliminarActividad(id);
                System.out.println("Actividad eliminada.");
            }
            case "3" -> {
                service.eliminarCliente(id);
                System.out.println("Cliente eliminado.");
            }
            default -> throw new IllegalStateException("Tipo de registro no reconocido.");
        }
    }

    private String readRecordType(String action) {
        while (true) {
            System.out.println("Selecciona el tipo de registro que quieres " + action + ":");
            System.out.println("1. Monitor");
            System.out.println("2. Actividad");
            System.out.println("3. Cliente");
            String type = readLine("Opción: ");
            if (type == null) {
                throw new IllegalArgumentException("Entrada finalizada.");
            }
            if (type.equals("1") || type.equals("2") || type.equals("3")) {
                return type;
            }
            System.out.println("Opción incorrecta. Elige 1, 2 o 3.");
        }
    }

    private boolean confirmar(String prompt) {
        String respuesta = readLine(prompt);
        return respuesta != null && respuesta.trim().equalsIgnoreCase("s");
    }

    private void printActividadDisponible(Actividad actividad) {
        System.out.printf("ID: %d | %s | %s | Monitor: %s | Plazas disponibles: %d%n",
                actividad.getId(), actividad.getNombre(), actividad.getFechaHora().format(DATE_TIME_FORMA
T),
                actividad.getMonitor().getNombre(), actividad.getPlazasDisponibles());
    }

    private String readRequiredText(String prompt) {
        while (true) {
            String value = readLine(prompt);
            if (value == null) {
                throw new IllegalArgumentException("Entrada finalizada.");
            }
            if (!value.isBlank()) {
                return value.trim();
            }
            System.out.println("Este campo no puede estar vacío.");
        }
    }

    private int readPositiveInt(String prompt) {
        while (true) {
            String value = readLine(prompt);
            if (value == null) {
                throw new IllegalArgumentException("Entrada finalizada.");
            }
            try {
```

PARTE: src/main/java/es/activa/main/ActivaApplication.java (continuación)

```
                int number = Integer.parseInt(value);
                if (number > 0) {
                    return number;
                }
            } catch (NumberFormatException ignored) {
                // Show the same clear validation message for non-numeric and non-positive values.
            }
            System.out.println("Introduce un número entero positivo.");
        }
    }

    private long readPositiveLong(String prompt) {
        while (true) {
            String value = readLine(prompt);
            if (value == null) {
                throw new IllegalArgumentException("Entrada finalizada.");
            }
            try {
                long number = Long.parseLong(value);
                if (number > 0) {
                    return number;
                }
            } catch (NumberFormatException ignored) {
                // Show the same clear validation message for non-numeric and non-positive values.
            }
            System.out.println("Introduce un identificador numérico positivo.");
        }
    }

    private LocalDateTime readDateTime(String prompt) {
        while (true) {
            String value = readLine(prompt);
            if (value == null) {
                throw new IllegalArgumentException("Entrada finalizada.");
            }
            try {
                return LocalDateTime.parse(value, DATE_TIME_FORMAT);
            } catch (DateTimeParseException exception) {
                System.out.println("Fecha u hora no válida. Usa el formato dd/MM/yyyy HH:mm.");
            }
        }
    }

    private LocalDate readDate(String prompt) {
        while (true) {
            String value = readLine(prompt);
            if (value == null) {
                throw new IllegalArgumentException("Entrada finalizada.");
            }
            try {
                return LocalDate.parse(value, DATE_FORMAT);
            } catch (DateTimeParseException exception) {
                System.out.println("Fecha no válida. Usa el formato dd/MM/yyyy.");
            }
        }
    }

    private String readLine(String prompt) {
        System.out.print(prompt);
        return scanner.hasNextLine() ? scanner.nextLine() : null;
    }

    private static String usefulMessage(Throwable exception) {
        String message = exception.getMessage();
        return message == null || message.isBlank() ? exception.getClass().getSimpleName() : message;
    }

    private static void printMenu() {
        System.out.println("""
                === ACTIVA · Gestión de actividades ===
                1. Crear monitor
                2. Crear actividad
                3. Buscar actividades disponibles
                4. Crear cliente
                5. Crear reserva
                6. Cancelar reserva
                7. Listar actividades
                8. Modificar monitor, actividad o cliente
                9. Eliminar monitor, actividad o cliente
```

PARTE: src/main/java/es/activa/main/ActivaApplication.java (continuación)

```
                10. Salir
                """);
    }
}
```